
XR Trainer

StretchSense Glove Pipeline

Technical Documentation

Real-time hand-tracking capture, validation, 3D visualisation, recording, and data export.

Package	xr_hand (version 0.1.0)
Runtime	Python 3.10 or newer
Platform	Windows / PowerShell (cross-platform Python core)
Transport	OSC over UDP, 127.0.0.1:9002
Capture rate	~60 Hz stream; configurable save rate

Contents

	Section	Description
1	System Overview	How the components fit together
2	Architecture & Data Flow	End-to-end pipeline stages
3	Installation	Environment setup on Windows
4	Quick Start	Common workflows in one place
5	Command Reference	Every script and its options
6	The Joint Model	26-joint layout and skeleton
7	Wire Format	OSC packet structure on the network
8	Module Reference	What each source file does
9	Data Formats	JSONL, CSV, and Excel outputs
10	Validation & Stream Health	Error and warning semantics
11	Mock Mode & Fault Injection	Running without hardware
12	Testing	Automated and manual checks
13	Troubleshooting	Common problems and fixes

1. System Overview

This project is a real-time pipeline for capturing, inspecting, and exporting hand-tracking data from a StretchSense Reality Glove driven by the XR Trainer application. The glove streams skeletal pose data over the network; the pipeline receives it, checks it for integrity, reconstructs the hand in 3D, and lets you record sessions for later review and analysis.

Each hand is described by 26 joints following the OpenXR XR_HAND_JOINT layout. Every joint carries a local position and an orientation quaternion, updated roughly 60 times a second.

What you can do with it

- **Watch the hand live** in an interactive 3D viewer while the glove streams.
- **Record sessions** to disk and scrub through them later like a video.
- **Validate the stream** for dropped, frozen, malformed, or NaN-laden packets.
- **Export** any time window to a tidy, formatted spreadsheet, or a full session to CSV.
- **Develop without hardware** using a built-in mock generator with injectable faults.

Design principles

- **Numeric-only downstream:** string header fields are normalised to placeholders at the receiver, so the validator and parser only ever see numbers.
- **Thread isolation:** the network server runs on its own thread and hands packets to the viewer through a bounded, thread-safe queue.
- **Swappable rendering:** the viewer is the only surface that touches matplotlib, so the backend can be replaced without changing the scripts.
- **One canonical wire layout:** the mock generator emits exactly the same 187-value payload the real glove does, so the whole pipeline is exercised identically.

2. Architecture & Data Flow

Data moves through the pipeline in a fixed sequence of stages. Each stage has a single responsibility and a narrow interface to the next.

```

glove / XR Trainer
  |  OSC over UDP  (127.0.0.1:9002)
  v
[ receiver ]  bind socket, normalise strings, queue raw values
  |
  v
[ validator ]  length / NaN / zero / quaternion checks per packet
               StreamMonitor tracks counter freezes & gaps
  |
  v
[ parser ]    187 values  ->  HandFrame (26 Joint records)
  |
  +-----+-----+
  v               v
[ kinematics ]   [ recorder ]
local -> world FK  append frame to JSONL
  |               |
  v               v
[ viz3d / playback ]  recordings/*.jsonl
interactive 3D viewer  |
                       v
                       [ export ] -> CSV / XLSX

```

Stage responsibilities

Stage	Module	Responsibility
Receive	receiver.py	Bind the UDP socket, dispatch by OSC address, queue raw numeric payloads.
Validate	validator.py	Reject structurally broken packets; warn on suspicious values and counter behaviour.
Parse	parser.py	Turn a flat value list into a typed HandFrame with 26 named joints.
Kinematics	kinematics.py	Chain parent-relative transforms into world-space joint positions.
Visualise	viz3d.py / playback_viewer.py	Draw the skeleton live, or scrub a recording with a data panel.
Record	recorder.py	Write validated frames to JSONL, optionally downsampled.
Export	export.py / export_xlsx.py	Convert recordings to CSV or a formatted Excel workbook.

3. Installation

Requirements

- Python 3.10 or newer.
- The XR Trainer application and a StretchSense Reality Glove for live capture (not required for mock mode).

Dependencies

Installed automatically by the step below. Core runtime packages:

Package	Purpose
numpy	Vector / matrix maths for forward kinematics.
matplotlib	Interactive 3D viewer and playback UI.
python-osc	OSC server that receives UDP packets from XR Trainer.
openpyxl	Writes the formatted Excel export.
pytest (dev)	Test runner for the automated suite.

Setup (Windows / PowerShell)

```
cd "C:\Users\nkim2\Desktop\xr trainer"  
python -m venv .venv  
.\.venv\Scripts\Activate.ps1  
pip install -e ".[dev]"
```

Installing with `-e` puts the package in editable mode, so source edits take effect without reinstalling.

If activation is blocked

If `Activate.ps1` is blocked by execution policy, run this once per machine, then re-open the shell:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

4. Quick Start

With XR Trainer running and the glove streaming to 127.0.0.1:9002, the workflows below cover most day-to-day use.

One-command record + playback + export

The recommended path. Records a clip, opens it for review, then writes a CSV:

```
.\scripts\record_and_export.ps1          # 10 s default, 5 Hz
.\scripts\record_and_export.ps1 -Duration 30 # record 30 s
.\scripts\record_and_export.ps1 -Hz 10     # save 10 frames/sec
```

Produces recordings\realglove_YYYYMMDD_HHMMSS.jsonl and a matching .csv.

Record a session manually

```
python scripts/record.py
```

Starts immediately and runs until you press Ctrl+C, saving to recordings\session_<id>.jsonl and printing the playback command. Add --mock to record without the glove.

Scrub a recording like a video

```
python scripts/playback.py recordings\session_<id>.jsonl
```

Opens the 3D hand on the left and a per-joint data panel on the right. Full controls are described in section 5.

Live viewer only

```
python scripts/run_osc.py          # live 3D viewer
python scripts/run_osc.py --record recordings/my.jsonl # view + record
python scripts/run_osc.py --dump --no-viz          # diagnose stream, no GUI
```

Without the glove (mock)

```
python scripts/run_mock.py          # animated mock viewer
python scripts/record.py --mock --duration 10 # mock recording
python scripts/test_faults.py        # headless fault checks
```

5. Command Reference

run_osc.py — live pipeline

Receiver, validator, parser, and 3D viewer wired together for the real glove.

Option	Default	Meaning
--host	127.0.0.1	Interface to bind (0.0.0.0 accepts any)
--port	9002	UDP port XR Trainer sends to
--address	/v1/animation/	OSC address of the kinematic stream
--dump	off	Log one line per received packet
--no-viz	off	Validate only, no GUI window
--record PATH	none	Also write each frame to a .jsonl file
--duration S	none	Auto-close the viewer after S seconds
--hz N	5.0	Frames saved per second (0 = keep all)

record.py — headless recorder

Writes validated frames to JSONL with no viewer. Supports both live OSC and mock sources.

Option	Default	Meaning
--mock	off	Use the mock generator instead of OSC
--port	9002	UDP port for live capture
--output PATH	auto	Output file (default recordings/session_*)
--duration S	none	Stop after S seconds (else Ctrl+C)
--hz N	5.0	Frames saved per second (0 = keep all)

playback.py — scrubbable viewer

Loads a whole recording and lets you move through it.

Control	Action
Frame slider / arrow keys	Step through the clip one frame at a time.
Select (range) slider	Pick a time window for playback and export.
Play / Pause	Auto-advance within the selected window.
Export selection -> Excel	Write just the selected frames to a formatted .xlsx.

The data panel shows both hands' world-space joint positions and highlights the joints that moved into the current frame.

export.py — JSONL to CSV

Option	Default	Meaning
input	required	Path to a .jsonl recording
--output PATH	same name	Output .csv path
--hand left right	both	Filter to a single hand

Helper scripts

Script	Purpose
run_mock.py	Full mock pipeline with an animated viewer; edit toggles to inject faults.
test_faults.py	Headless check that each injected fault is detected as expected.
record_and_export.ps1	One-command record + playback + export wrapper.

6. The Joint Model

Each hand has 26 joints, indexed 0-25 in the OpenXR order. The wrist (index 1) is the root of the skeleton; the palm and the five finger chains hang off it. Bone connectivity is defined as parent-child index pairs used both for drawing and for forward kinematics.

Joint indices

Idx	Joint	Idx	Joint
0	PALM	13	MIDDLE_INTERMEDIATE
1	WRIST	14	MIDDLE_DISTAL
2	THUMB_METACARPAL	15	MIDDLE_TIP
3	THUMB_PROXIMAL	16	RING_METACARPAL
4	THUMB_DISTAL	17	RING_PROXIMAL
5	THUMB_TIP	18	RING_INTERMEDIATE
6	INDEX_METACARPAL	19	RING_DISTAL
7	INDEX_PROXIMAL	20	RING_TIP
8	INDEX_INTERMEDIATE	21	LITTLE_METACARPAL
9	INDEX_DISTAL	22	LITTLE_PROXIMAL
10	INDEX_TIP	23	LITTLE_INTERMEDIATE
11	MIDDLE_METACARPAL	24	LITTLE_DISTAL
12	MIDDLE_PROXIMAL	25	LITTLE_TIP

Skeleton hierarchy

The wrist connects to the palm and to each finger's first joint. Every finger is a chain from metacarpal to tip:

```

WRIST -> PALM
WRIST -> THUMB_METACARPAL -> PROXIMAL -> DISTAL -> TIP
WRIST -> INDEX_METACARPAL -> PROXIMAL -> INTERMEDIATE -> DISTAL -> TIP
WRIST -> MIDDLE_METACARPAL -> PROXIMAL -> INTERMEDIATE -> DISTAL -> TIP
WRIST -> RING_METACARPAL -> PROXIMAL -> INTERMEDIATE -> DISTAL -> TIP
WRIST -> LITTLE_METACARPAL -> PROXIMAL -> INTERMEDIATE -> DISTAL -> TIP

```

Core data structures

Defined in `joints.py`. A `Joint` holds a name, a position, and an XYZW quaternion. A `HandFrame` bundles 26 joints with packet metadata.

```

Joint(name, x, y, z, qw, qx, qy, qz)

HandFrame(
    timestamp,          # tick counter, used time-like
    packet_counter,     # for freeze / gap detection

```

```
    hand_side,      # 'left' or 'right'  
    frame_id,  
    status,  
    joints,         # list of 26 Joint records  
)
```

7. Wire Format

XR Trainer sends to 127.0.0.1:9002 on the OSC address /v1/animation/kinematic/all with exactly 187 arguments per packet.

Packet layout (187 values)

Range	Count	Contents
[0]	1	Tick counter (int) — used as packet_counter and a time-like value.
[1]	1	Frame id (int).
[2]	1	Status / hand code (int).
[3]	1	Device-label string, e.g. "Reality Glove (L)" — encodes hand side.
[4]	1	Device-family string, e.g. "Reality Glove".
[5..186]	182	26 joints x 7 values: x, y, z, qx, qy, qz, qw.

Header total: 5 values. Joints: 26 x 7 = 182. Grand total: 187.

Per-joint value order

Each joint contributes seven floats in this order. Position is **parent-relative** (local space); the quaternion is **XYZW** with unit magnitude.

x	y	z	qx	qy	qz	qw
---	---	---	----	----	----	----

Hand-side resolution

The hand is determined from the device-label string at argument [3] ((L) or (R), or a trailing L/R). The receiver then replaces both string header fields with 0.0 placeholders so the validator and parser only handle numeric data.

Mock / legacy addresses

The receiver also accepts /hand/left and /hand/right with 187 numeric arguments, where the hand side comes from the address rather than a label string. This lets the mock sender drive the same receiver as the real glove.

8. Module Reference

Package `src/xr_hand/`

Module	Responsibility
<code>joints.py</code>	26 joint names, bone connectivity, and the Joint / HandFrame dataclasses.
<code>parser.py</code>	Convert a 187-value payload into a HandFrame; raises <code>PacketParseError</code> on bad input.
<code>validator.py</code>	Per-packet structural checks plus <code>StreamMonitor</code> for cross-packet counter health.
<code>receiver.py</code>	Threaded OSC/UDP server; dispatches by address and queues numeric payloads.
<code>kinematics.py</code>	<code>quat_to_mat3</code> and <code>forward_kinematics</code> : local transforms to world positions.
<code>mock.py</code>	<code>MockHandGenerator</code> producing canonical payloads with optional injected faults.
<code>recorder.py</code>	<code>FrameRecorder</code> : write/load JSONL, with optional per-hand downsampling.
<code>viz3d.py</code>	<code>HandViewer</code> : live matplotlib 3D skeleton with palm fill and fingertip markers.
<code>playback_viewer.py</code>	<code>PlaybackViewer</code> : scrubbable 3D + data panel + window selection and Excel export.
<code>export_xlsx.py</code>	<code>write_workbook</code> : tidy, styled Excel output with summary and joint-data sheets.

Scripts `scripts/`

Script	Responsibility
<code>run_osc.py</code>	Live real-glove pipeline with optional recording and auto-close.
<code>run_mock.py</code>	Live in-process mock pipeline; edit toggles to inject faults.
<code>record.py</code>	Headless recorder for live or mock sources.
<code>playback.py</code>	Open and scrub a recorded .jsonl session.
<code>export.py</code>	Convert a .jsonl recording to wide CSV.
<code>test_faults.py</code>	Headless fault-injection sanity check.
<code>record_and_export.ps1</code>	One-command record + playback + export pipeline.

Key interfaces

The receiver exposes a simple producer/consumer surface; the viewer drains the queue each animation tick:

```
rx = OSCHandReceiver(host, port, kinematic_addr=...)
rx.start()
for hand, raw in rx.drain(max_items=16):
    result = validate_raw_message(raw)
    if result.is_valid:
```

```
        frame = parse_hand_message(raw, hand_side_hint=hand)
        world = forward_kinematics(frame)
rx.stop()
```

9. Data Formats

JSONL recording

One JSON object per line — easy to inspect, grep, or stream. Each line is one hand frame:

```
{
  "wall_time": 1748123456.789,    # real clock time, for replay timing
  "timestamp": 4.233,             # OSC tick counter
  "packet_counter": 254,
  "hand_side": "right",
  "frame_id": 254,
  "status": 0,
  "joints": [
    {
      "name": "PALM", "x": 0.0, "y": 0.04, "z": 0.0,
      "qw": 1.0, "qx": 0.0, "qy": 0.0, "qz": 0.0},
    ...
  ]
}
```

Sample rate

Recordings are downsampled to 5 frames/sec by default (`--hz 5`). The glove still streams at ~60 Hz; only the saved frames are throttled, and each hand is throttled independently so both keep the full target rate. Every saved frame keeps its exact `wall_time` and `timestamp`. Pass `--hz 0` to keep every frame.

CSV export

One row per frame, wide layout. Positions and quaternions are kept in the device's native local frame; forward kinematics is left to the analyst. Columns:

```
wall_time, iso_time, timestamp, packet_counter,
hand_side, frame_id, status,
PALM_x, PALM_y, PALM_z, PALM_qw, PALM_qx, PALM_qy, PALM_qz,
WRIST_x, ... (26 joints x 7 columns = 182 columns)
```

Excel export

Written from the playback viewer's selected window. Unlike the wide CSV, the workbook is a tidy long table — one row per frame x hand x joint — with two sheets:

- **Summary:** source file, the selected window in elapsed and clock time, frame count, joints per hand, and the coordinate space used.
- **Joint data:** frame, elapsed seconds, clock, hand, joint, world XYZ, rotation quaternion (qw, qx, qy, qz), and a *moved* flag.

The sheet has a frozen, filterable header, per-hand row colouring (pale cyan for left, pale red for right), and bold joint names on rows that moved. World positions are used here because they move with the hand, alongside the raw quaternion that drives the motion.

10. Validation & Stream Health

Validation has two layers: per-packet structural checks, and cross-packet stream monitoring. Errors mean a packet is unusable and is dropped; warnings mean it is usable but something looks off and is logged.

Per-packet checks

Check	Severity	Condition
Length	Error	Argument count is not exactly 187.
Non-finite	Error	Any value is NaN or infinite.
All-zero joint	Warning	A joint's seven values are all ~0.
Quaternion magnitude	Warning	q deviates from 1.0 by more than 0.1.

Stream monitor (per hand)

Real-device tick counters legitimately repeat and skip within batched emissions, so only *persistent* freezes and *large* gaps are treated as problems. Tracked per hand by StreamMonitor:

Condition	Default threshold	Meaning
Counter freeze	20+ identical in a row	Stream likely stalled or stuck.
Counter gap	10+ ticks skipped	Packets dropped in transit.
Counter backwards	any decrease	Out-of-order or reset stream.

Defaults assume roughly a 100 Hz packet rate; adjust the thresholds if your stream rate differs substantially. A freeze emits one warning when it starts and one when the counter resumes, rather than spamming every packet.

11. Mock Mode & Fault Injection

MockHandGenerator emits the same 187-value payload the real glove does, so the entire pipeline runs identically without hardware. The hand animates through an open-to-fist curl driven by a slow sine. Each generator drives one hand; the two are offset along X so they do not overlap.

Fault toggles

Set these attributes on a generator to inject specific failure modes and confirm the validator catches them:

Toggle	Effect	Expected detection
stuck_fingertip = NAME	One tip stops moving (frozen offset).	None — visual-only, stream stays valid.
freeze_counter = True	Packet counter never increments.	StreamMonitor freeze warning.
zero_joint = NAME	One joint becomes all zeros.	All-zero joint warning.
wrong_length = True	Payload truncated by one value.	Length-mismatch error.
random_missing_prob = p	Each value has p chance of being NaN.	Non-finite error.

Trying it interactively

Edit the fault block near the top of `run_mock.py` and run it to watch a fault in the live viewer:

```
right_gen.stuck_fingertip = "INDEX_TIP"
# right_gen.freeze_counter = True
# right_gen.zero_joint = "MIDDLE_PROXIMAL"
# right_gen.wrong_length = True
# right_gen.random_missing_prob = 0.005
```


12. Testing

Automated suite

Run the full test suite with pytest:

```
pytest -q
```

Test file	Covers
test_pipeline.py	Parser correctness, per-packet validation, and StreamMonitor behaviour.
test_recorder.py	JSONL record/load round-trip fidelity.

Fault-injection check

A headless script runs the mock generator through the validator for each fault and reports whether the expected error or warning fired:

```
python scripts/test_faults.py
```

It verifies a clean stream stays clean, a stuck fingertip stays valid, and each injected fault (frozen counter, zero joint, wrong length, random NaN) is detected. It prints a per-check PASS/FAIL line and a final tally.

Manual verification

When first connecting XR Trainer, confirm the wire format before relying on the viewer:

```
python scripts/run_osc.py --dump --no-viz
```

This logs one line per packet so you can confirm the address pattern and argument count (187) match what the validator expects.

13. Troubleshooting

Symptom	Likely cause & fix
No packets / empty viewer	Confirm XR Trainer is streaming to 127.0.0.1:9002. Run with <code>--dump --no-viz</code> to see whether any OSC packets arrive at all.
"unexpected arg count" warnings	The stream is not sending 187 arguments on the expected address. Check the <code>--address</code> value and the XR Trainer output format.
"could not determine hand" skips	The device-label string at arg [3] does not contain (L)/(R) or a trailing L/R. Verify the glove label in XR Trainer.
Counter-frozen warnings	The stream stalled (20+ identical counters). Check the glove connection and XR Trainer status; a brief freeze that resumes is logged and is usually fine.
Activate.ps1 is blocked	Run <code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code> once, then reopen the shell.
Quaternion-magnitude warnings	Joint orientation deviates from unit length by more than 0.1 — usually a transient; persistent cases suggest a data issue upstream.
Recording grows slowly	Frames are downsampled to 5 Hz by default. Pass <code>--hz 0</code> to keep every frame, or a higher value for more detail.
Excel export looks empty	Make sure a window is selected on the range slider before clicking Export; only the selected frames are written.

End of document. For the authoritative behaviour of any component, consult the corresponding module in `src/xr_hand/`.